

Базовые сущности Kubernetes: Service,Ingress

Microservice Architecture

REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?





Майоров Антон

Старший разработчик ПО

- 10+ лет опыта разработки ПО на платформе .NET
- Специализация: приложения со сложной бизнес логикой
- Последние 4 года - старший системный разработчик в Ренессанс Страхование

@amaiorov

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе

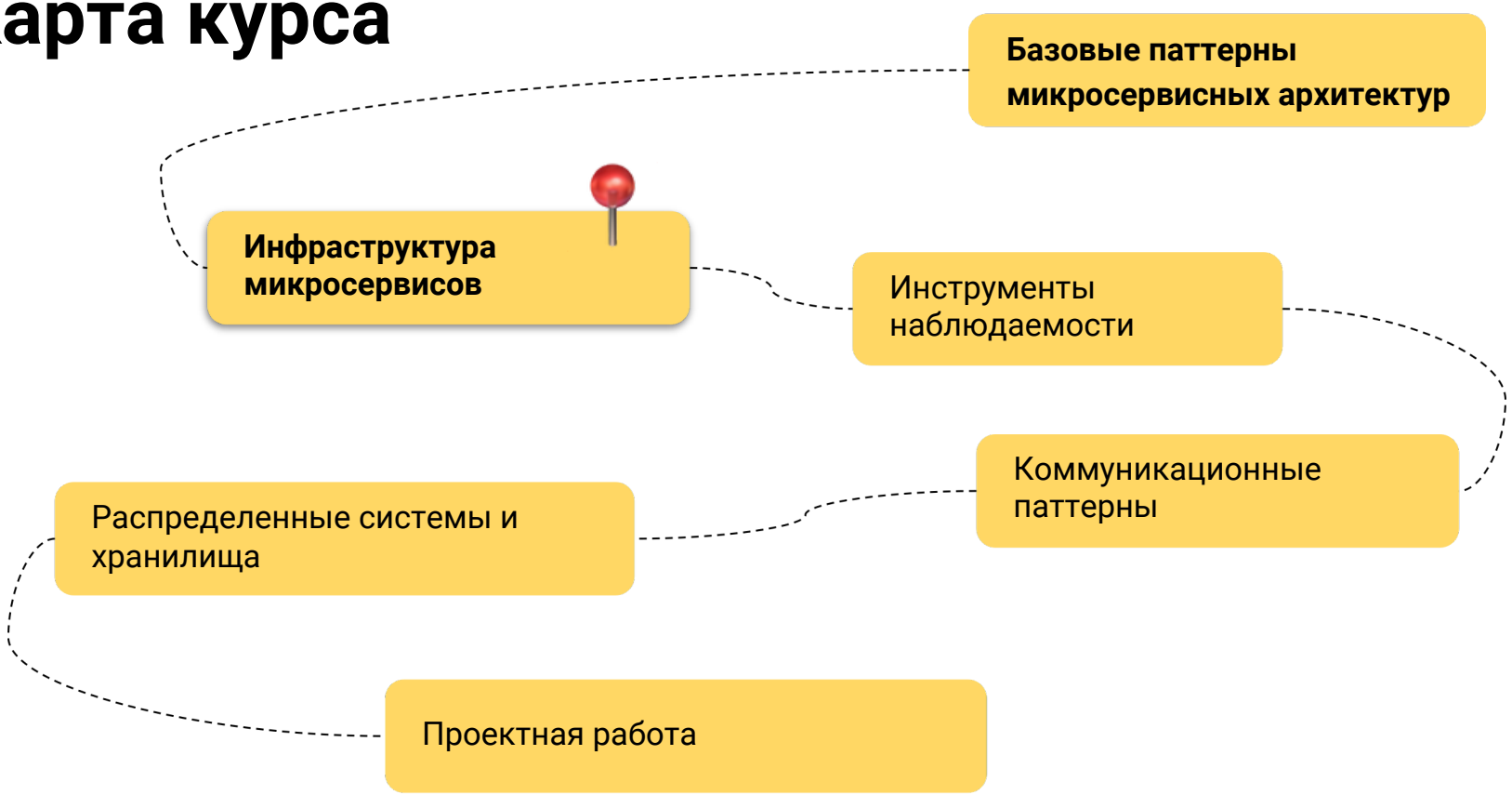


Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса


Базовые паттерны микросервисных архитектур
Инфраструктура микросервисов
Инструменты наблюдаемости
Коммуникационные паттерны
Распределенные системы и хранилища
Проектная работа

Маршрут вебинара

- Service
- Ingress
- Подведем итоги

Цели вебинара

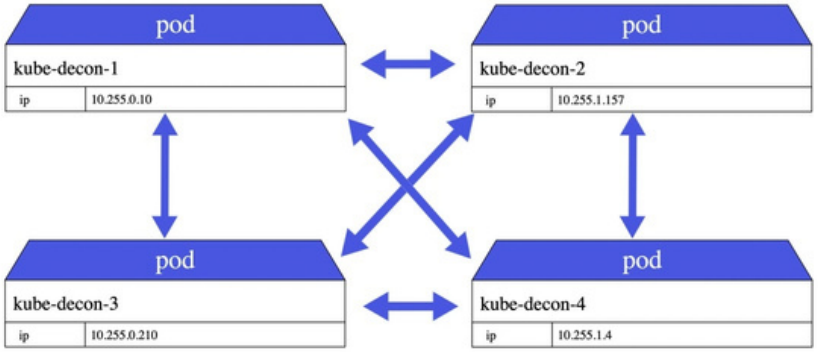
К концу занятия вы сможете

1. Настроить стратегию обновления для приложения
2. Настроить сетевой доступ к вашему приложению как внутри кластера так и снаружи

Service

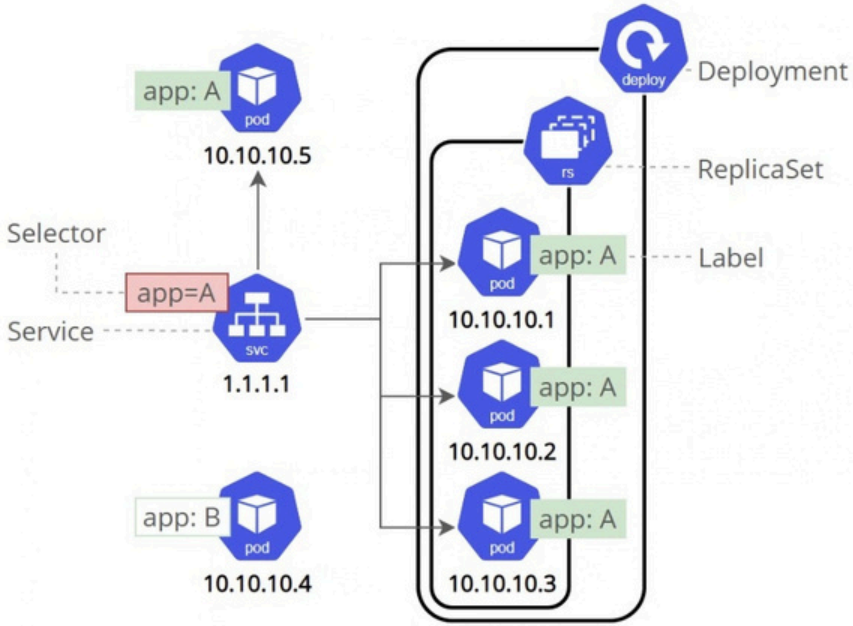
Зачем вообще нужен Service?

Но ведь у пода же есть уникальный IP, по которому он доступен с любой ноды кластера?



- **ненадежность IP адреса пода**–при пересоздании пода IP ему присваивается каждый раз динамически
- **балансировка**–одно приложение может быть развернуто в любом количестве подов, и запросы к приложению необходимо балансировать различным образом
- **динамичность**–количество реплик приложения (подов) может динамически изменяться, необходимо успевать вносить изменения в конфигурацию балансировщика

Принцип работы Service

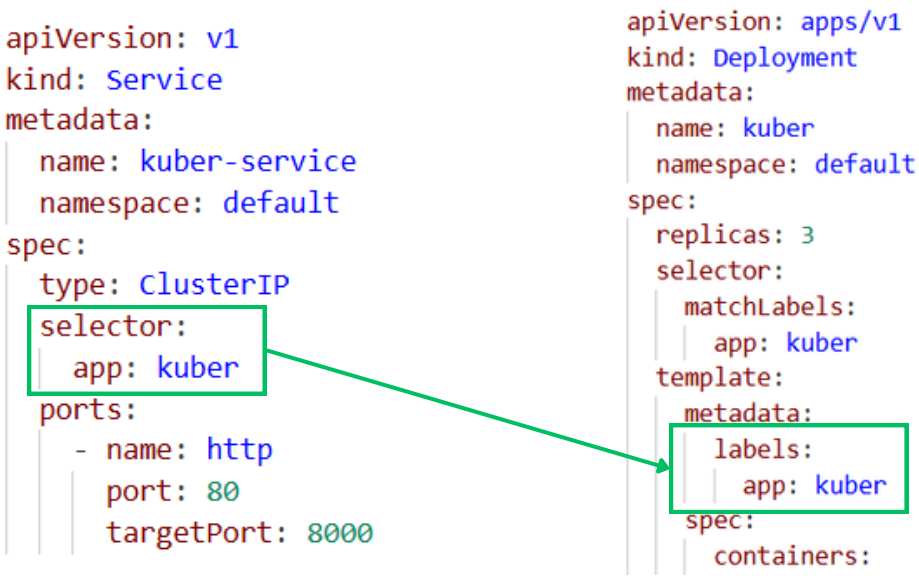


- Объект Kubernetes, который позволяет обращаться **к нескольким подам** через свое имя и IP адрес через механизм меток и селекторов
- Отслеживает **изменения конфигурации** запущенных подов и динамически вносит правки в таблицу маршрутизации
- Работает на **транспортном уровне** сети (L4)

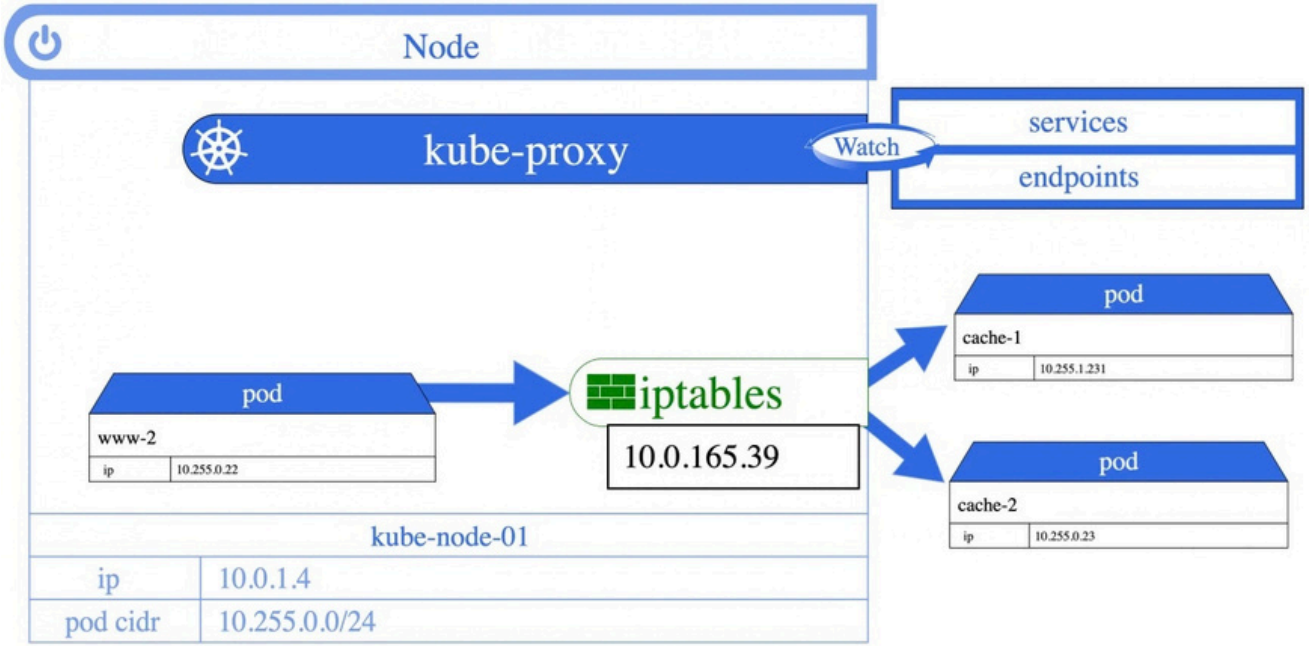
Service. Пример манифеста

```yaml
apiVersion: v1
kind: Service
metadata:
  name: kuber-service
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: kuber
  ports:
    - name: http
      port: 80
      targetPort: 8000
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kuber
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: kuber
  template:
    metadata:
      labels:
        app: kuber
    spec:
      containers:
```

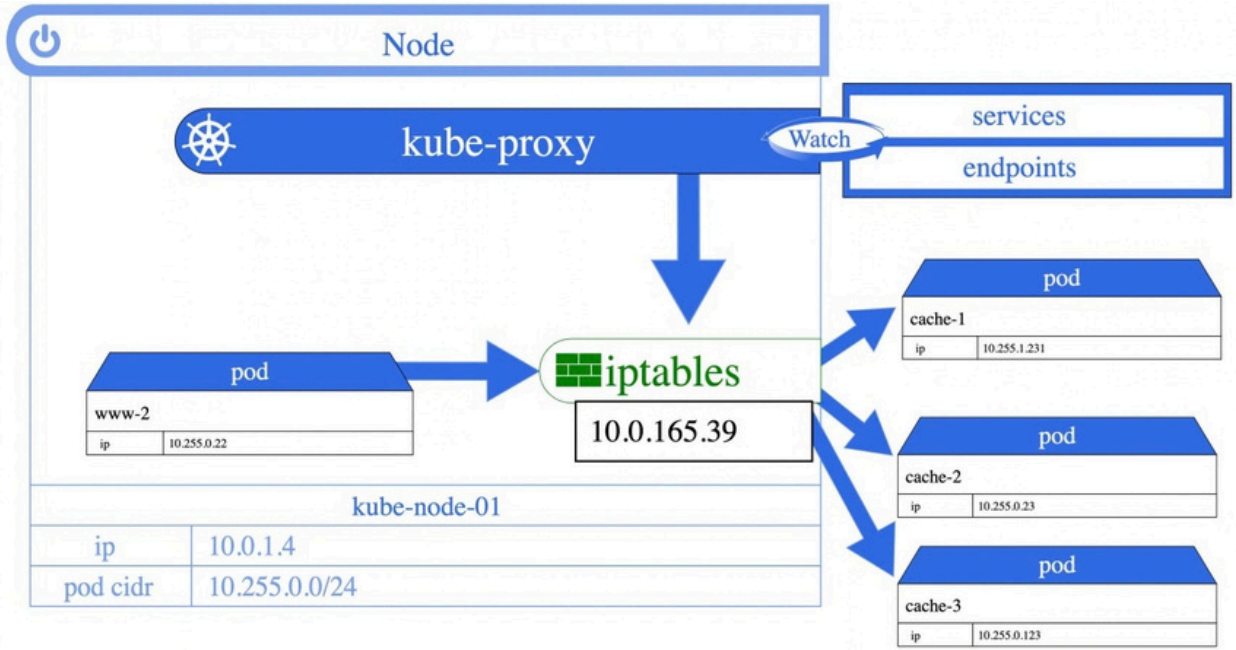


Service. Работа с IP tables



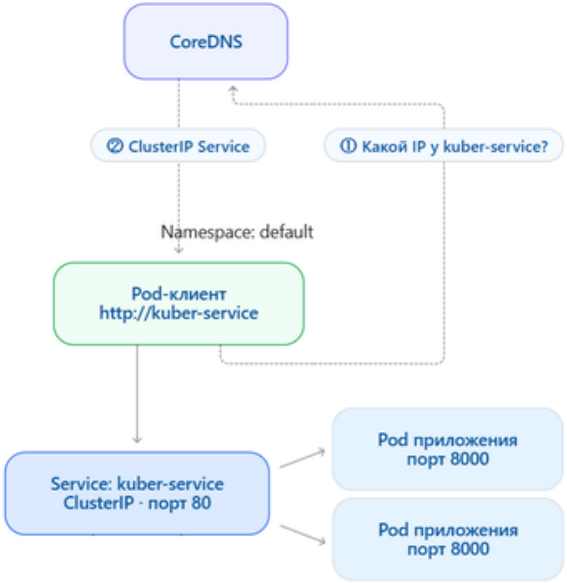
http://kube-decon.carson-anderson.com/

Service. Работа с IP tables



http://kube-decon.carson-anderson.com/

DNS: доступ по имени



Имя Service известно заранее, поэтому клиентам не нужно знать его IP

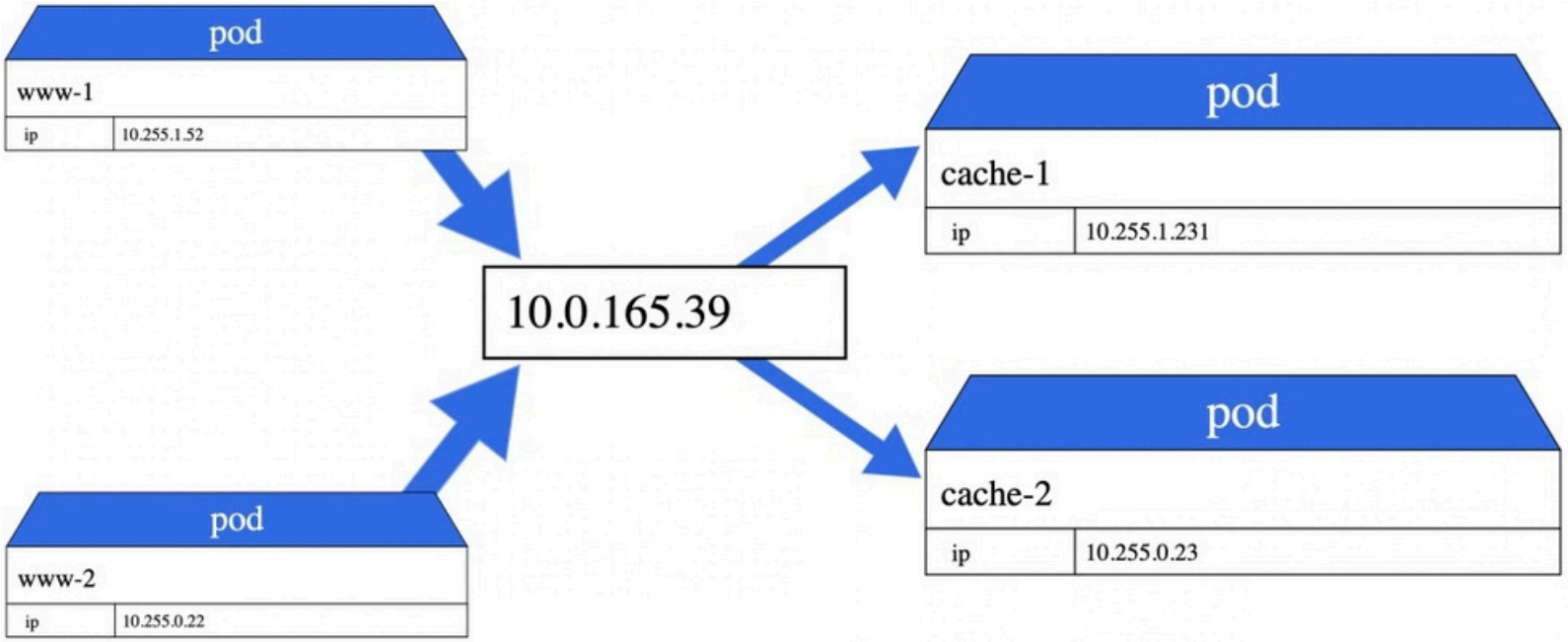
```yaml
apiVersion: v1
kind: Service
metadata:
  name: kuber-service
  namespace: default
```

Обращение	Пример
В своём namespace	kuber-service
С указанием namespace	kuber-service.default
Полное имя	kuber-service.default.svc.cluster.local

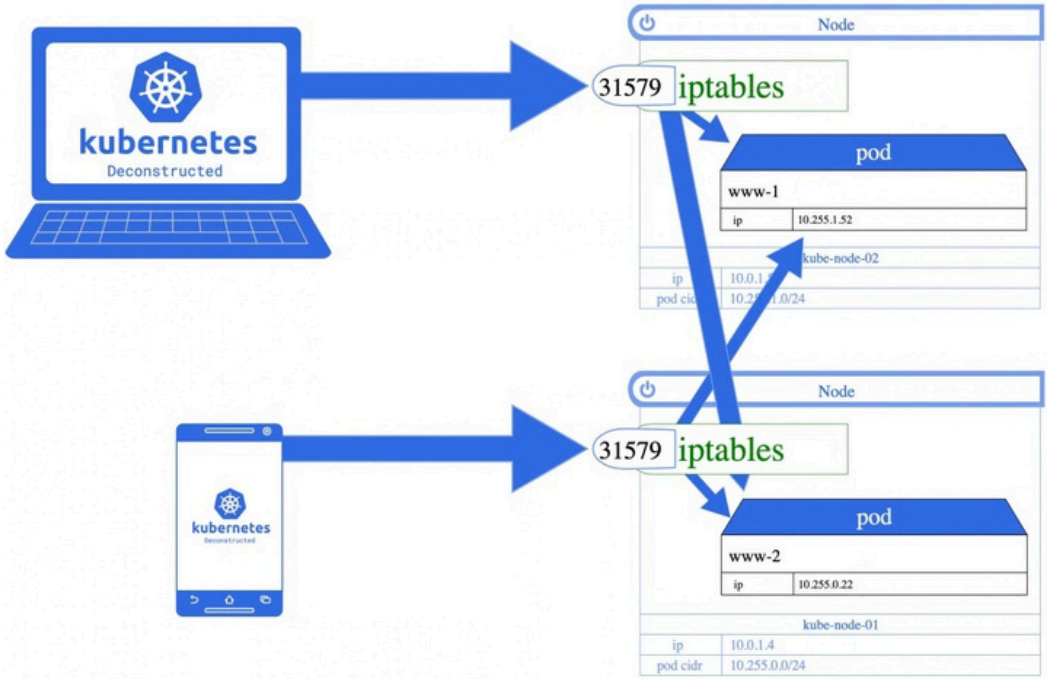
Типы объекта Service

- **ClusterIP**–по-умолчанию, для внутрикластерного взаимодействия между приложениями
- **NodePort**–позволяет организовать простейший доступ к приложению в кластере снаружи. Открывает определенный порт на всех нодах кластера, через который можно обратиться к приложению
- **LoadBalancer**–позволяет настроить доступ к приложению извне кластера. Для реализации необходим внешний по отношению к кластеру балансировщик.
- **ExternalName**-описывает абстракцию внешнего по отношению к кластеру ресурса как внутрикластерный сервис

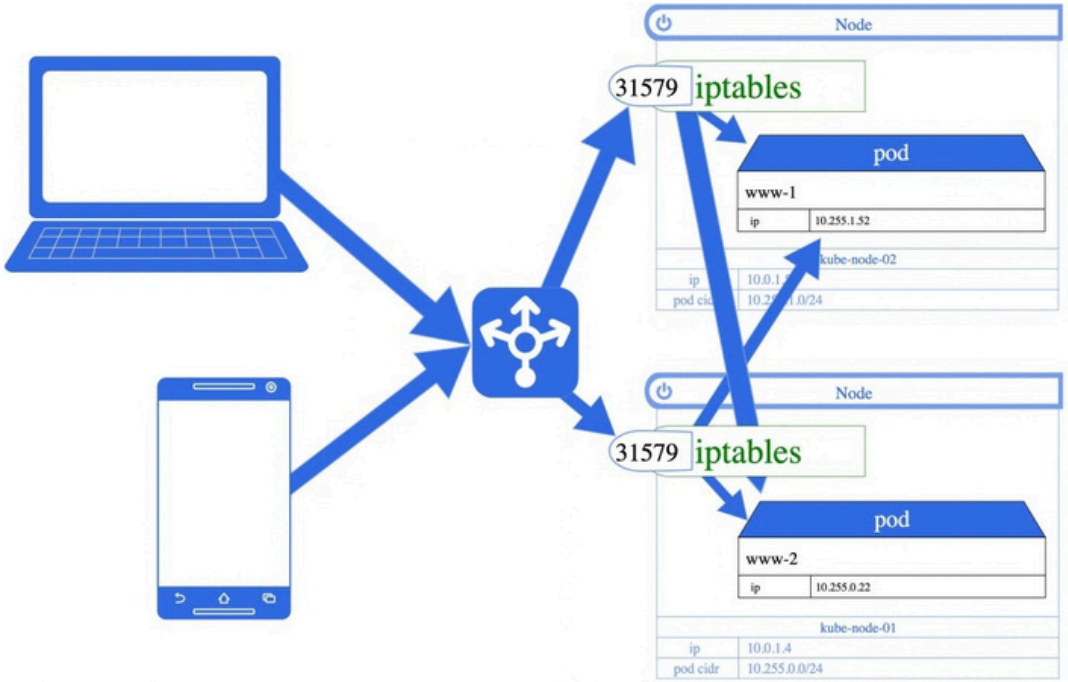
Service. ClusterIP



Service. NodePort



Service. LoadBalancer



Headless service

- ClusterIP –None
- сервис не получает IP адреса, по которому можно было бы к нему обратиться
- но позволяет выполнить resolve IP адресов подов, которыми управляет сервис
- необходимо для случаев, когда балансировка трафика не требуется или нужен более сложный алгоритм балансировки, чем позволяет задать Kubernetes
- наиболее частые сценарии использования
  - работа с Stateful-приложениями – например с БД
  - балансировка трафика для grpc приложений

Вопросы?

+ если есть вопросы

− если вопросов нет

Ingress

Зачем он вообще нужен



есть же NodePort и LoadBalancer сервисы

Ingress

Service:

- **только L4**–сервисы работают на транспортном уровне, никаких VirtualHosts и StickySessions
- **открывать порт на каждой ноде под каждое приложение –дорого и неудобно**
- **выделять отдельный IP под каждый сервис в случае с LoadBalancer -дорого**

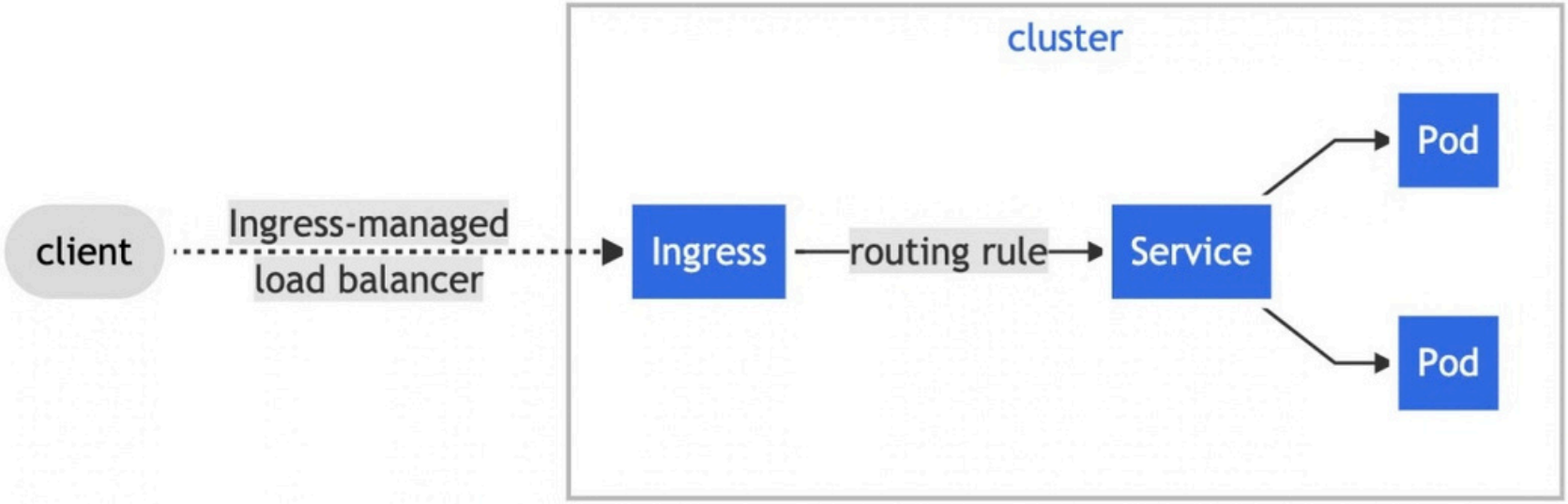
Зачем он вообще нужен

- Объект kubernetes, который описывает как должен маршрутизироваться внешний трафик
- Нет встроенного контроллера –необходимо поставить, чаще всего это nginx ingress, но есть множество других
- работает на L7
- Маршрутизирует трафик на сервисы внутри кластера –т.е он не заменяет, а дополняет объект service
- Работает с http и https протоколами
- Будущее за GatewayAPI

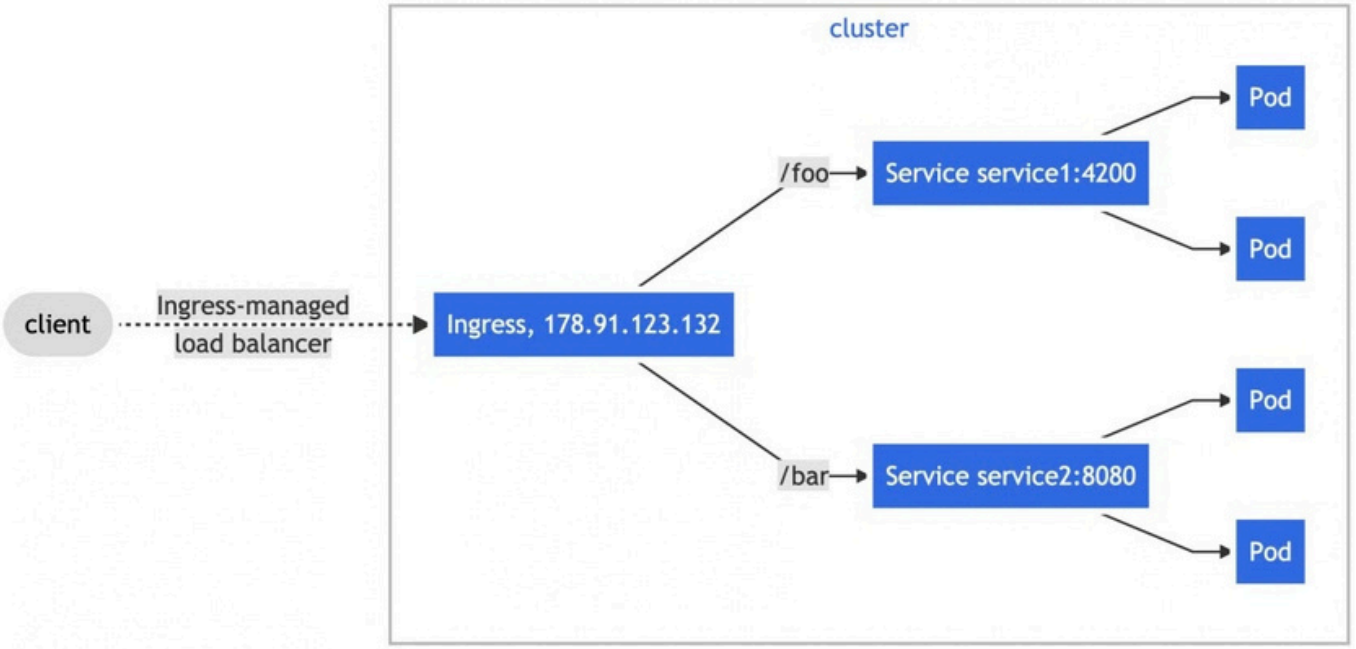
Ingress. Пример манифеста

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```

Ingress. Принцип работы



Ingress. Принцип работы



Вопросы?

+ если есть вопросы

− если вопросов нет

Демо

Рефлексия

Вопросы для повторения

1. Какие проблемы решает Service в Kubernetes и почему недостаточно обращаться напрямую к IP-адресам Pod?
2. Как Service выбирает Pod для передачи трафика и какую роль играют метки, селекторы и namespace?
3. Чем отличаются port и targetPort в настройках Service?
4. Как организована балансировка трафика через Service и как готовность Pod влияет на получение запросов?
5. Чем отличаются типы Service ClusterIP, NodePort, LoadBalancer и ExternalName и когда их используют?
6. Как работает обращение к Service по DNS-имени внутри кластера и какую роль выполняет CoreDNS?
7. Что такое Headless Service, чем он отличается от обычного ClusterIP и для каких задач нужен?
8. Для чего нужен Ingress, чем он отличается от Ingress Controller и как взаимодействует с Service при маршрутизации HTTP/HTTPS-трафика?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
иберемихвработу ✏️❤️